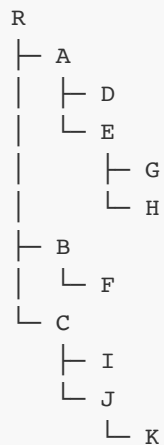


第六章作业答案

1

解答：

树形（缩进）表示如下：



括号表示法为：

```
R(A(D,E(G,H)),B(F),C(I,J(K)))
```

根结点到最深叶结点的路径可以是 R -> A -> E -> G、R -> A -> E -> H 或 R -> C -> J -> K，均包含 4 个结点。因此，按题目规定，该树的高度为 4。

递归求树高的伪代码如下：

```
height(node):
    if node == null:
        return 0
    maxChildHeight = 0
    for each child in node.children:
        maxChildHeight = max(maxChildHeight, height(child))
    return maxChildHeight + 1
```

“长子—右兄弟”转换中，每个结点的左指针指向它的第一个孩子，右指针指向它的下一个兄弟。非空关系如下：

```

R.left  = A
A.left  = D      A.right = B
D.right = E
E.left  = G
G.right = H
B.left  = F      B.right = C
C.left  = I
I.right = J
J.left  = K

```

转换后二叉树的先序遍历序列为：

```
R A D E G H B F C I J K
```

这与原有序树的先根遍历序列相同。

2

解答：

前 H 层，即第 0 层到第 $H - 1$ 层必须全部填满；第 H 层至少有一个结点、至多有 K^H 个结点。因此：

$$(K^H - 1)/(K - 1) + 1 \leq N \leq (K^{(H+1)} - 1)/(K - 1)$$

当 $H = 0$ 时，上式两端都等于 1。

采用从 0 开始的层序编号时，编号为 n 的非根结点 ($n > 0$) 的父结点编号为：

$$\text{parent}(n) = \text{floor}((n - 1) / K)$$

编号为 n 的结点的第 i 个子结点的候选编号为：

$$\text{child}(n, i) = K*n + i, \quad i = 1, 2, \dots, K$$

只有当 $K*n + i < N$ 时，该子结点才实际存在。

对于非根结点 n ，它在父结点的孩子序列中的位置为：

$$\text{position} = ((n - 1) \bmod K) + 1$$

因此，它有右兄弟必须同时满足：

$$\begin{aligned} (n - 1) \bmod K &\neq K - 1 \\ n + 1 &< N \end{aligned}$$

第一项表示它不是父结点的第 k 个孩子，第二项用于排除完全树最后一个实际结点之后的空位置。若右兄弟存在，其编号为 $n + 1$ 。

3

解答：

按题目规定的同规模合并规则，各次关键合并如下：

```
union(0, 1):  parent[1] = 0
union(2, 3):  parent[3] = 2
union(4, 5):  parent[5] = 4
union(6, 7):  parent[7] = 6
union(0, 2):  parent[2] = 0
union(4, 6):  parent[6] = 4
union(8, 9):  parent[9] = 8
union(10, 11): parent[11] = 10
union(8, 10): parent[10] = 8
```

执行 `find(3)` 时，路径 `3 -> 2 -> 0` 被压缩，所以 `parent[3] = 0`。

`union(6, 8)` 合并两个规模均为 `4` 的集合。根据同规模规则，把结点 `8` 所在树的根挂到结点 `6` 所在树的根之下，因此 `parent[8] = 4`。

执行 `union(3, 5)` 时，结点 `3` 所在集合规模为 `4`，结点 `5` 所在集合规模为 `8`，故令 `parent[0] = 4`。随后：

- `find(1)` 将路径 `1 -> 0 -> 4` 压缩，使 `parent[1] = 4`。
- `find(11)` 将路径 `11 -> 10 -> 8 -> 4` 压缩，使 `parent[11] = 4`、`parent[10] = 4`。

没有经过相应查找路径的结点仍可间接指向根。最终数组为：

```
下标：    0  1  2  3  4  5  6  7  8  9 10 11
parent:  4  4  0  0  4  4  4  6  4  8  4  4
```

其中 `4` 是全部 `12` 个元素的最终根。

按规模合并时，只有当一个结点所在的树被挂到另一棵规模不小于它的树下面，该结点的深度才可能增加 `1`。每增加一次深度，它所在新树的规模至少变为原来的 `2` 倍。若某结点深度为 `h`，其所在树至少含有 `2h` 个结点，所以：

```
2h <= N
h <= log2(N)
```

因此树高为 `O(log N)`。